Soluciones: práctica de repaso de Bash (sesión 6)

Oct 21, 2026 · @Angela

Ejercicio 1. inventario_cuentas.sh

```
#!/bin/bash
# inventario_cuentas.sh - Cuentas de /etc/passwd que usan una shell
determinada

if [ $# -gt 1 ]; then
    echo "Error: demasiados parámetros"
    echo "Uso: $0 [shell]"
    exit 1
fi

# Si no se indica la shell, se usa /bin/bash
shell_buscada="/bin/bash"
if [ $# -eq 1 ]; then
    shell_buscada="$1"
fi

# Ruta absoluta: una o más veces "/" seguida de un nombre en minúsculas
if [[ ! $shell_buscada =~ ^(/[a-z]+)+$ ]]; then
    echo "Error: $shell_buscada no es una ruta válida"
    exit 1
fi

total=0
uid_altos=0

while IFS=':' read -r usuario x uid gid desc home shell; do
    if [ "$shell" = "$shell_buscada" ]; then
        echo "$usuario:$uid:$home:$shell"
        ((total++))
        if [ "$uid" -ge 1000 ]; then
            ((uid_altos++))
        fi
    fi
done < /etc/passwd
```

```
echo "----"
echo "Cuentas listadas: $total"
echo "Con UID >= 1000: $uid_altos"
```

Prueba

```
$ ./inventario_cuentas.sh
root:0:/root:/bin/bash
ubuntu:1000:/home/ubuntu:/bin/bash
postgres:102:/var/lib/postgresql:/bin/bash
----
Cuentas listadas: 3
Con UID >= 1000: 1
$ ./inventario_cuentas.sh bash
Error: bash no es una ruta válida
```

Ejercicio 2. backup_etc.sh

```
#!/bin/bash
# backup_etc.sh - Copia comprimida de /etc con registro en el sistema

# Muestra el mensaje con la hora y lo registra con la etiqueta aso-backup
registrar() {
    echo "$(date +%H:%M:%S) $1"
    logger -t aso-backup "$1"
}

if [ $# -gt 2 ]; then
    registrar "Error: demasiados parámetros. Uso: $0 [destino] [máximo]"
    exit 1
fi

# Valores por defecto; se cambian si llegan los parámetros
destino="/var/backups"
maximo=7

if [ $# -ge 1 ]; then
    destino="$1"
fi

if [ $# -eq 2 ]; then
    maximo="$2"
fi
```

```
if [ ! -d "$destino" ]; then
    registrar "Error: el directorio $destino no existe"
    exit 1
fi

# Primero el formato (regex) y después el valor (comparación numérica)
if [[ ! $maximo =~ ^[0-9]+$ ]]; then
    registrar "Error: $maximo no es un número"
    exit 1
fi

if [ "$maximo" -le 0 ]; then
    registrar "Error: el número máximo de copias debe ser mayor que 0"
    exit 1
fi

fichero="$destino/etc-$(date +%Y%m%d-%H%M%S).tar.gz"

if tar -czf "$fichero" /etc 2> /dev/null; then
    registrar "Copia creada: $fichero"
else
    registrar "Error: no se ha podido crear la copia $fichero"
    exit 1
fi

# Solo cuentan los ficheros con el formato etc-AAAAMMDD-HHMMSS.tar.gz
copias=$(ls "$destino" | grep -cE '^etc-[0-9]{8}-[0-9]{6}\.tar\.gz$')

if [ "$copias" -gt "$maximo" ]; then
    registrar "Aviso: hay $copias copias y el máximo recomendado es $maximo"
fi
```

Alternativa para contar las copias. En lugar de `ls | grep -cE`, se puede recorrer el directorio con un bucle y comprobar cada nombre con `=~`. El resultado es el mismo:

```
# Solo cuentan los ficheros con el formato etc-AAAAMMDD-HHMMSS.tar.gz
copias=0
for nombre in $(ls "$destino"); do
    if [[ $nombre =~ ^etc-[0-9]{8}-[0-9]{6}\.tar\.gz$ ]]; then
        ((copias++))
    fi
done
```

Con `grep` la regex va entre comillas simples porque es un argumento del comando; con `=~` va sin comillas.

Otra forma de recorrer el directorio. En lugar de `$(ls "$destino")`, el bucle puede recorrer `"$destino"/*`. Bash sustituye el `*` por los ficheros del directorio, sin ejecutar ningún comando:

```
# Solo cuentan los ficheros con el formato etc-AAAAMMDD-HHMMSS.tar.gz
copias=0
for ruta in "$destino"/*; do
    if [[ $ruta =~ /etc-[0-9]{8}-[0-9]{6}\.tar\.gz$ ]]; then
        ((copias++))
    fi
done
```

La diferencia es que ahora la variable contiene la ruta completa (`/tmp/copias/etc-20261021-201504.tar.gz`) y no solo el nombre. Por eso la regex no empieza por `^etc-`, sino por `/etc-`: la barra marca que el nombre del fichero empieza justo ahí.

Prueba (con una copia antigua y un fichero `otra.txt` ya en el directorio)

```
$ sudo ./backup_etc.sh /tmp/copias 1
20:15:04 Copia creada: /tmp/copias/etc-20261021-201504.tar.gz
20:15:04 Aviso: hay 2 copias y el máximo recomendado es 1
$ sudo ./backup_etc.sh /tmp/copias siete
20:15:04 Error: siete no es un número
$ sudo ./backup_etc.sh /tmp/copias 0
20:15:04 Error: el número máximo de copias debe ser mayor que 0
$ journalctl -t aso-backup
```

Ejercicio 3. gestion_usuarios.sh

```
#!/bin/bash
# gestion_usuarios.sh - Prepara el alta o la baja de un usuario del alumnado

# Muestra el error y la forma de uso, y termina
error() {
    echo "Error: $1"
    echo "Uso: $0 alta | baja identificador"
    exit 1
}

# Recibe nombre, primer apellido y segundo apellido; devuelve aluXXYYZ
generar_id() {
    nn=$(echo "$1" | cut -c1-2)
    a1=$(echo "$2" | cut -c1-2)
    a2=$(echo "$3" | cut -c1)
```

```
    echo "alu$nn$a1$a2" | tr 'A-Z' 'a-z'
}

# Devuelve éxito si el usuario aparece al principio de una línea de
/etc/passwd
existe_usuario() {
    grep -q "^$1:" /etc/passwd
}

alta() {
    read -p "Nombre: " nombre
    read -p "Apellidos: " apellidos

    if [[ ! $nombre =~ ^[A-Za-z]+$ ]]; then
        error "el nombre debe ser una sola palabra formada por letras"
    fi

    # El espacio dentro de [[ ]] se escapa: dos palabras separadas por un
espacio
    if [[ ! $apellidos =~ ^[A-Za-z]+\ [A-Za-z]+$ ]]; then
        error "escribe los dos apellidos separados por un espacio"
    fi

    apellido1=$(echo "$apellidos" | cut -d' ' -f1)
    apellido2=$(echo "$apellidos" | cut -d' ' -f2)
    id=$(generar_id "$nombre" "$apellido1" "$apellido2")
    echo "Identificador generado: $id"

    if existe_usuario "$id"; then
        error "el usuario $id ya existe"
    fi

    echo "Para crearlo: sudo useradd -m -s /bin/bash -c \"$nombre
$apellidos\" $id"
}

baja() {
    id="$1"

    if [[ ! $id =~ ^alu[a-z]{5}$ ]]; then
        error "$id no tiene el formato aluXXYYZ"
    fi

    if ! existe_usuario "$id"; then
        error "el usuario $id no existe"
    fi
```

```
    echo "Para eliminarlo: sudo userdel -r $id"
}

# Programa principal
if [ $# -eq 0 ]; then
    error "falta la acción"
fi

if [ "$1" = "alta" ]; then
    alta
elif [ "$1" = "baja" ]; then
    if [ -z "$2" ]; then
        error "falta el identificador"
    fi
    baja "$2"
else
    error "la acción $1 no es válida"
fi
```

Alternativa para generar el identificador. En lugar de `cut`, se pueden extraer los caracteres con subcadenas `${variable:inicio:longitud}`. La posición de inicio empieza en 0. El resto del script no cambia:

```
# Recibe nombre, primer apellido y segundo apellido; devuelve aluXXYYZ
generar_id() {
    nn=${1:0:2}
    a1=${2:0:2}
    a2=${3:0:1}
    echo "alu$nn$a1$a2" | tr 'A-Z' 'a-z'
}
```

Con `"Lucia"`, `${1:0:2}` devuelve `Lu`: 2 caracteres a partir de la posición 0.

Prueba

```
$ ./gestion_usuarios.sh alta
Nombre: Lucia
Apellidos: Ferrer Mas
Identificador generado: alulufem
Para crearlo: sudo useradd -m -s /bin/bash -c "Lucia Ferrer Mas" alulufem
$ ./gestion_usuarios.sh baja root
Error: root no tiene el formato aluXXYYZ
Uso: ./gestion_usuarios.sh alta | baja identificador
$ ./gestion_usuarios.sh baja
Error: falta el identificador
Uso: ./gestion_usuarios.sh alta | baja identificador
```

Solución con solo las dos funciones obligatorias. El enunciado pide como mínimo una función para generar el identificador y otra para mostrar los errores. En esta versión solo están esas dos: el alta y la baja se resuelven dentro del `if` del programa principal, y la comprobación de si el usuario existe se hace directamente con `grep -q`. La salida es la misma que en la prueba anterior.

```
#!/bin/bash
# gestion_usuarios.sh - Prepara el alta o la baja de un usuario del alumnado

# Muestra el error y la forma de uso, y termina
error() {
    echo "Error: $1"
    echo "Uso: $0 alta | baja identificador"
    exit 1
}

# Recibe nombre, primer apellido y segundo apellido; devuelve aluXXYYZ
generar_id() {
    nn=$(echo "$1" | cut -c1-2)
    a1=$(echo "$2" | cut -c1-2)
    a2=$(echo "$3" | cut -c1)
    echo "alu$nn$a1$a2" | tr 'A-Z' 'a-z'
}

# Programa principal
if [ $# -eq 0 ]; then
    error "falta la acción"
fi

if [ "$1" = "alta" ]; then
    read -p "Nombre: " nombre
    read -p "Apellidos: " apellidos
```

```
    if [[ ! $nombre =~ ^[A-Za-z]+$ ]]; then
        error "el nombre debe ser una sola palabra formada por letras"
    fi

    # El espacio dentro de [[ ]] se escapa: dos palabras separadas por un
espacio
    if [[ ! $apellidos =~ ^[A-Za-z]+\ [A-Za-z]+$ ]]; then
        error "escribe los dos apellidos separados por un espacio"
    fi

    apellido1=$(echo "$apellidos" | cut -d' ' -f1)
    apellido2=$(echo "$apellidos" | cut -d' ' -f2)
    id=$(generar_id "$nombre" "$apellido1" "$apellido2")
    echo "Identificador generado: $id"

    if grep -q "^$id:" /etc/passwd; then
        error "el usuario $id ya existe"
    fi

    echo "Para crearlo: sudo useradd -m -s /bin/bash -c \"$nombre
$apellidos\" $id"

elif [ "$1" = "baja" ]; then
    id="$2"

    if [ -z "$id" ]; then
        error "falta el identificador"
    fi

    if [[ ! $id =~ ^alu[a-z]{5}$ ]]; then
        error "$id no tiene el formato aluXXYYZ"
    fi

    if ! grep -q "^$id:" /etc/passwd; then
        error "el usuario $id no existe"
    fi

    echo "Para eliminarlo: sudo userdel -r $id"

else
    error "la acción $1 no es válida"
fi
```

Ejercicio 4. informe_procesos.sh

```
#!/bin/bash
# informe_procesos.sh - Procesos del usuario actual ordenados por consumo de
CPU

if [ $# -gt 1 ]; then
    echo "Error: demasiados parámetros"
    echo "Uso: $0 [N]"
    exit 1
fi

n="$1"

# Si se ha pasado N, se comprueba el formato y el valor
if [ -n "$n" ]; then
    if [[ ! $n =~ ^[0-9]+$ ]]; then
        echo "Error: $n no es un número válido"
        exit 1
    fi
    if [ "$n" -le 0 ]; then
        echo "Error: el número debe ser mayor que 0"
        exit 1
    fi
fi

# Listado completo, de mayor a menor %CPU, guardado en procesos.txt
ps -u "$USER" -o pid,pcpu,pmem,comm --sort=-pcpu > procesos.txt

if [ -n "$n" ]; then
    # N + 1 líneas: la cabecera y los N primeros procesos
    head -n $((n + 1)) procesos.txt
else
    cat procesos.txt
fi

# El total no cuenta la línea de cabecera
lineas=$(wc -l < procesos.txt)
total=$((lineas - 1))

echo "----"
echo "Procesos de $USER: $total"
```

Prueba

```
$ ./informe_procesos.sh 3
  PID %CPU %MEM COMMAND
 2154 12.4  6.8 firefox
 1587  3.1  2.9 gnome-shell
 2890  1.2  0.4 gnome-terminal-
----
Procesos de alumno: 58
$ ./informe_procesos.sh tres
Error: tres no es un número válido
```